

Week4

Task1:

Process Synchronization and Child Process Monitoring

Using waitpid()

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid, result;
    int status;

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child process started. PID = %d\n", getpid());
        sleep(2);
        printf("Child process completed.\n");
        exit(5);
    }

    printf("Parent process. PID = %d\n", getpid());
    printf("Waiting for child process...\n");

    result = waitpid(pid, &status, 0);

    if (result == -1) {
        perror("waitpid");
        return 1;
    }

    if (WIFEXITED(status)) {
        printf("Child exited normally with status = %d\n",
               WEXITSTATUS(status));
    } else if (WIFSIGNALED(status)) {
        printf("Child terminated by signal = %d\n",
               WTERMSIG(status));
    }

    printf("Parent process completed.\n");

    return 0;
}
```

Output:

```
Parent process. PID = 583
Child process started. PID = 584
Waiting for child process...
Child process completed.
Child exited normally with status = 5
Parent process completed.
```

Task2:

To Retrieve PATH Variable, Parse Search Directories, Locate Executables, Verify Permissions, Handle Missing Commands, Test Command Resolution

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

int main(int argc, char *argv[]) {
    char *path;
    char *path_copy;
    char *dir;
    char fullpath[PATH_MAX];
    int found = 0;

    if (argc != 2) {
        printf("Usage: %s <command>\n", argv[0]);
        return 1;
    }

    path = getenv("PATH");

    if (path == NULL) {
        printf("PATH variable not found.\n");
        return 1;
    }

    printf("PATH = %s\n\n", path);

    path_copy = strdup(path);

    if (path_copy == NULL) {
        perror("strdup");
        return 1;
    }

    dir = strtok(path_copy, ":");

    while (dir != NULL) {
        snprintf(fullpath, sizeof(fullpath), "%s/%s", dir, argv[1]);

        if (access(fullpath, F_OK) == 0) {
```

```

    if (access(fullpath, F_OK) == 0) {
        printf("File found: %s\n", fullpath);

        if (access(fullpath, X_OK) == 0) {
            printf("Execute permission: YES\n");
            printf("Command resolved successfully.\n");
            found = 1;
            break;
        } else {
            printf("Execute permission: NO\n");
        }
    }

    dir = strtok(NULL, ":");
}

if (!found) {
    printf("Command '%s' not found in PATH.\n", argv[1]);
}

free(path_copy);

return 0;
}

```

Output:

```

root@DESKTOP-VMN932K:~# ./code2 pwd
PATH = /usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/
Ubuntu_2404.1.68.0_x64_79rhkp1fndgsc:/mnt/c/Program Files/Common Files/Oracle/Java/javapa
m Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/Windows/system32:/mnt/c/Windows:/mr
/c/Windows/System32/OpenSSH:/mnt/c/Program Files/MySQL/MySQL Server 8.0/bin:/mnt/c/Progra
ELL/AppData/Local/Programs/Python/Launcher:/mnt/c/Users/DELL/AppData/Local/Microsoft/Wind
bin:/mnt/c/Microsoft VS Code/bin:/mnt/c/Users/DELL/AppData/Roaming/npm:/snap/bin

```

Task3:

To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX 1024

void expand_variables(char *input, char *output) {
    int i = 0;
    int j = 0;

    while (input[i] != '\0' && j < MAX - 1) {

        if (input[i] == '$') {
            char variable[100];
            int k = 0;
            int start = i;

            i++;

            if (input[i] == '{') {
                i++;

                while (input[i] != '\0' && input[i] != '}' &&
                    k < 99) {
                    variable[k++] = input[i++];
                }

                if (input[i] == '}')
                    i++;
            } else if (isalpha((unsigned char)input[i]) ||
                input[i] == '_') {

                while ((isalnum((unsigned char)input[i]) ||
                    input[i] == '_') && k < 99) {
                    variable[k++] = input[i++];
                }
            }
        }
    }
}
```

```

        input[i] == '_' ) && k < 99) {
            variable[k++] = input[i++];
        }
    } else {
        output[j++] = input[start];
        continue;
    }
    variable[k] = '\0';
    char *value = getenv(variable);
    if (value != NULL) {
        int m = 0;

        while (value[m] != '\0' && j < MAX - 1)
            output[j++] = value[m++];
    } else {
        printf("Warning: Undefined variable: %s\n", variable);
    }
    } else {
        output[j++] = input[i++];
    }
}
output[j] = '\0';
}

int main() {
    char input[MAX];
    char output[MAX];
    printf("Enter a string: ");
    if (fgets(input, sizeof(input), stdin) == NULL)
        return 1;
    input[strcspn(input, "\n")] = '\0';
    expand_variables(input, output);
    printf("Original : %s\n", input);
    printf("Expanded : %s\n", output);
    return 0;
}

```

Output:

```

Enter a string: Hello
Original : Hello
Expanded : Hello
root@DESKTOP-VMN932K:~# ./code31
Enter a string: hello $abc
Warning: Undefined variable: abc
Original : hello $abc
Expanded : hello

```

Task4:

To Identify Built-ins, Create Dispatch Table, Execute In Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#define MAX_INPUT 1024
#define MAX_ARGS 100

void builtin_pwd(char **args) {
    char cwd[MAX_INPUT];

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("pwd");
}

void builtin_cd(char **args) {
    if (args[1] == NULL) {
        char *home = getenv("HOME");
        if (home != NULL)
            chdir(home);
        else
            printf("HOME not set\n");
    } else {
        if (chdir(args[1]) != 0)
            perror("cd");
    }
}

void builtin_echo(char **args) {
    int i = 1;
    while (args[i] != NULL) {
        printf("%s", args[i]);
        if (args[i + 1] != NULL)
            printf(" ");
        i++;
    }
    printf("\n");
}
```

```

}
void execute_command(char **args) {
    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return;
    }
    if (pid == 0) {
        execvp(args[0], args);
        printf("Command not found: %s\n", args[0]);
        exit(1);
    } else {
        waitpid(pid, NULL, 0);
    }
}
int main() {
    char input[MAX_INPUT];
    char *args[MAX_ARGS];
    char *token;
    while (1) {
        printf("myshell> ");
        fflush(stdout);
        if (fgets(input, sizeof(input), stdin) == NULL)
            break;
        input[strcspn(input, "\n")] = '\0';
        if (strlen(input) == 0)
            continue;
        int argc = 0;
        token = strtok(input, " ");
        while (token != NULL && argc < MAX_ARGS - 1) {
            args[argc++] = token;
            token = strtok(NULL, " ");
        }
        args[argc] = NULL;
        if (strcmp(args[0], "exit") == 0) {
            printf("Exiting shell...\n");
            exit(0);
        }
    }
}

```

```

        input[strcspn(input, "\n")] = '\0';
        if (strlen(input) == 0)
            continue;
        int argc = 0;
        token = strtok(input, " ");
        while (token != NULL && argc < MAX_ARGS - 1) {
            args[argc++] = token;
            token = strtok(NULL, " ");
        }
        args[argc] = NULL;
        if (strcmp(args[0], "exit") == 0) {
            printf("Exiting shell...\n");
            exit(0);
        }
        if (strcmp(args[0], "pwd") == 0) {
            builtin_pwd(args);
        } else if (strcmp(args[0], "cd") == 0) {
            builtin_cd(args);
        } else if (strcmp(args[0], "echo") == 0) {
            builtin_echo(args);
        } else {
            execute_command(args);
        }
    }
    return 0;
}

```

Output:

```

myshell> pwd
/root
myshell> cd /tmp
myshell> abc
Command not found: abc
myshell> exit
Exiting shell...

```